PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL — LABORATÓRIO DE REDES DE COMPUTADORES

INTERCEPTAÇÃO E DEFESA DE UM PROTOCOLO DE APLICAÇÃO

Fase 1: reconhecimento e escuta passiva

Bernardo Garcia Gabriel Vieira Guilherme Cavazzotto — Porto Alegre, 2026

Objetivo

Demonstrar na prática por que um protocolo de aplicação sem proteção quebra a confidencialidade. Para isso construímos um serviço alvo que se comunica em texto claro e um interceptador passivo que lê esse tráfego direto da rede, sem participar da comunicação.

Serviço alvo: protocolo KVP

O serviço alvo é o KVP (*Key-Value Protocol*), um repositório chave-valor com login. É orientado a linha, usa UTF-8 e roda sobre TCP na porta 5050, sem cifragem e sem *hash* de senha, portanto tudo trafega legível. As credenciais vão no comando `LOGIN` e os dados nos comandos `SET` e `GET`. As credenciais de teste são o usuário `admin` com a senha `s3nh4-pucrs`.

Comando	Efeito
`LOGIN <usuario> <senha>`	Autentica a sessão (credenciais em texto claro).
`SET <chave> <valor>`	Grava um valor.
`GET <chave>`	Lê um valor.
`DEL <chave>`	Remove um valor.
`LIST`	Lista as chaves.
`PING` / `QUIT`	Testa a conexão / encerra a sessão.

Interceptador passivo (sniffer)

O sniffer usa um socket *raw* (`AF_PACKET`, `SOCK_RAW`), sem biblioteca de captura pronta. Ele recebe os quadros Ethernet da interface, faz o *parsing* manual de Ethernet, IPv4 e TCP, filtra a porta 5050 e imprime o conteúdo da aplicação, destacando as linhas sensíveis (`LOGIN`, `SET`, `GET`, `VALUE`). Ele apenas escuta, não envia nada: um observador totalmente passivo já lê tudo o que passa.

Ambiente de teste

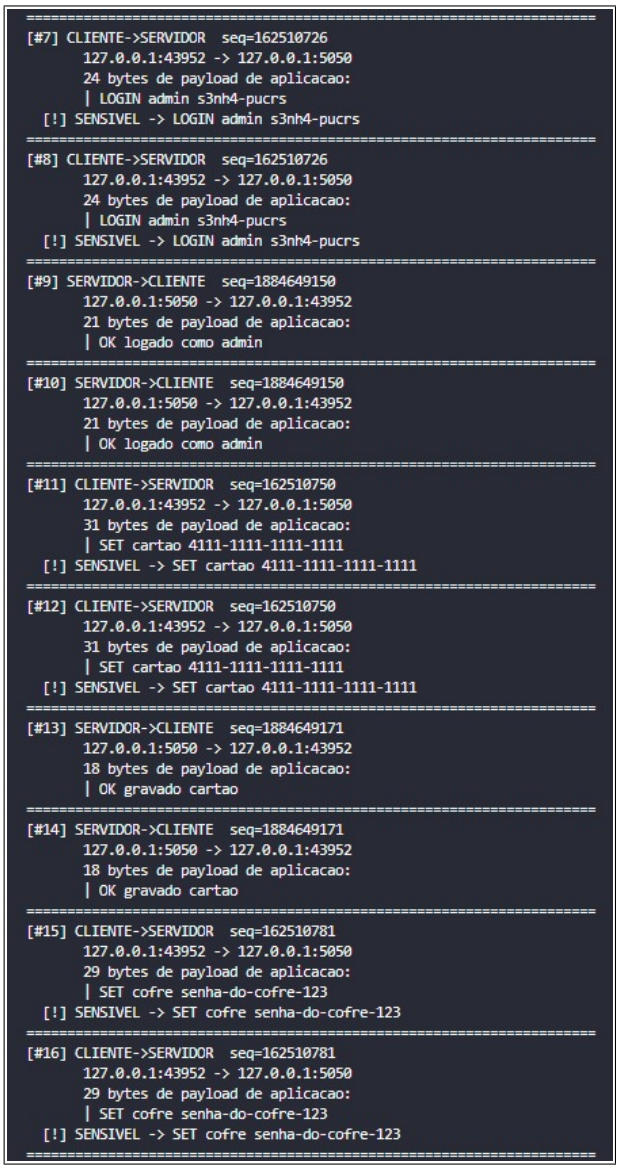
O teste foi feito em um único host, com duas captações independentes do mesmo tráfego: o Wireshark (via `tshark`) na interface `lo0` do macOS, e o sniffer próprio em um Linux (contêiner,

kernel 6.12) na interface `lo`, pois `AF_PACKET` só existe no Linux. O conteúdo é idêntico ao do cenário de três VMs (VM-cliente, VM-servidor e VM-observador) descrito no `README.md`. O cliente roda em modo demonstração e envia, em sequência: `PING`, `LOGIN admin s3nh4-pucrs`, `SET cartao 4111-1111-1111-1111`, `SET cofre senha-do-cofre-123`, `GET cartao`, `LIST` e `QUIT`.

Evidências de captura

A Figura 1 mostra parte da saída do sniffer próprio. A senha do `LOGIN`, o número de cartão e a senha do cofre aparecem legíveis, lidos por um processo que nunca conversou com o servidor. Cada *payload* aparece duas vezes (por exemplo, os pacotes 7 e 8 com o mesmo `LOGIN`): é o comportamento do *loopback* no Linux, onde o `AF_PACKET` vê o pacote na saída e na entrada da interface `lo`; em um segmento Ethernet real cada pacote apareceria uma vez.

Figura 1 – Saída do sniffer próprio com o conteúdo em texto claro.



As Figuras 2 e 3 mostram o mesmo tráfego no Wireshark. Na primeira, o pacote 19 está selecionado e o painel de bytes exibe `LOGIN admin s3nh4-pucrs` em ASCII. Na segunda, a função *Follow TCP Stream* remonta a conexão inteira, com o cliente em vermelho e o servidor em azul.

Figura 2 – Pacote 19 com o comando LOGIN em texto claro (Wireshark).

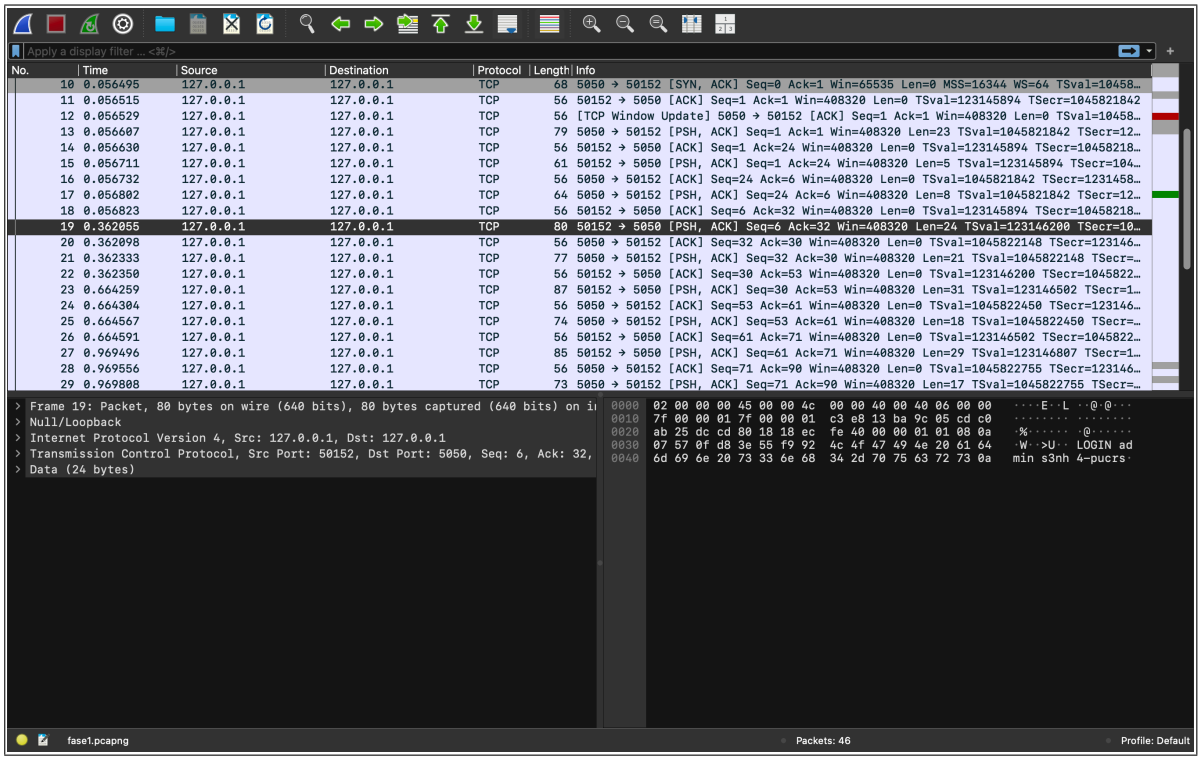
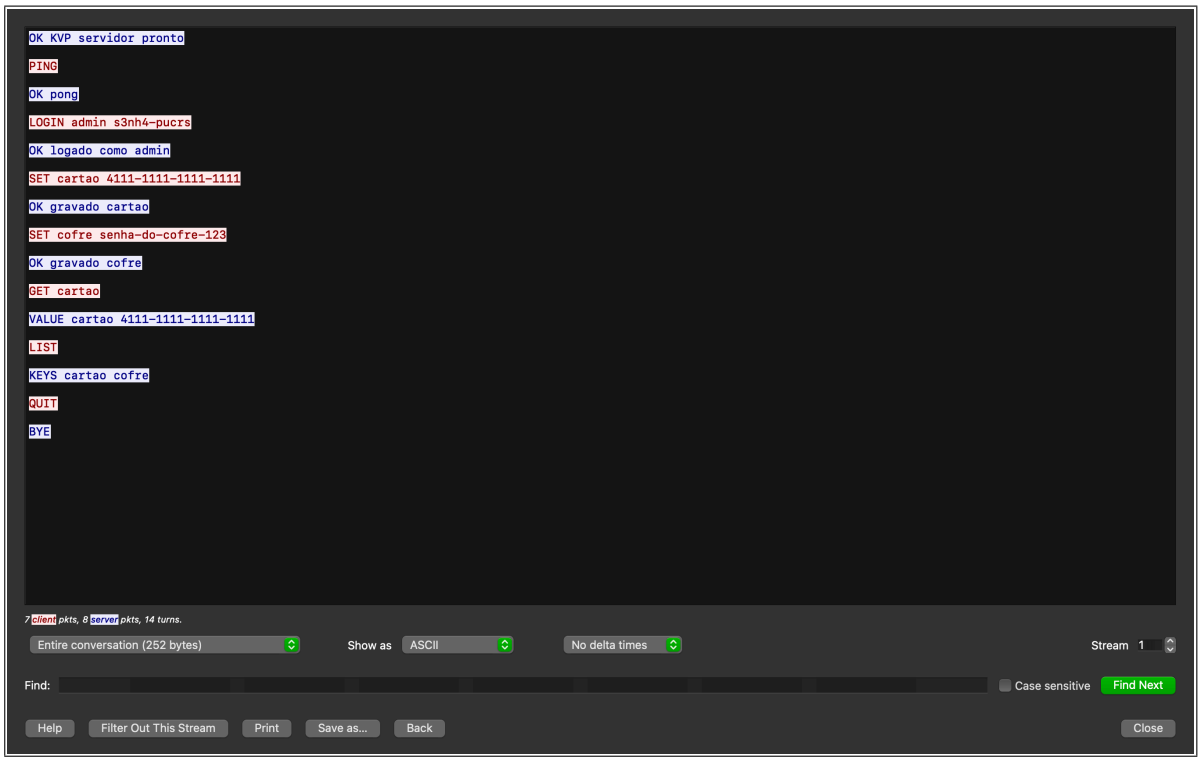


Figura 3 – Conversa completa reconstruída (Follow TCP Stream).



A Tabela 1 lista os pacotes com conteúdo de aplicação da conexão `127.0.0.1:50152 -> 127.0.0.1:5050`, com o número de cada pacote e o texto observado. As linhas em negrito são as de dado sensível.

Tabela 1 – Pacotes com conteúdo legível na captura do Wireshark.

Pacote	Sentido	Conteúdo observado
19	cliente → servidor	**`LOGIN admin s3nh4-pucrs`**
21	servidor → cliente	`OK logado como admin`
23	cliente → servidor	**`SET cartao 4111-1111-1111-1111`**
27	cliente → servidor	**`SET cofre senha-do-cofre-123`**
31	cliente → servidor	`GET cartao`
33	servidor → cliente	**`VALUE cartao 4111-1111-1111-1111`**
37	servidor → cliente	`KEYS cartao cofre`

Conclusão

As duas ferramentas leem as credenciais e os valores em texto claro, sem nenhum esforço de decodificação. A causa é a ausência de cifragem no canal (sem TLS): qualquer byte enviado vira texto visível, e o atacante passivo não precisa autenticar nem enviar nada. Na Fase 3, o canal receberá TLS e as senhas passarão a usar *hash*; com isso, o sniffer desta fase deixará de conseguir ler o conteúdo.